Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

2. Phân tích hướng domain — DDD & Bounded Contexts . . . 2
2.1. Conway's Law — tổ chức quyết định kiến trúc . . . 3
2.2. DDD cơ bản — ngôn ngữ chung và các building blocks . . . 5
2.2.1. Ubiquitous Language — ngôn ngữ chung . . . 5
2.2.2. Các building blocks . . . 6
2.2.3. DDD dành cho kỹ sư phần mềm: tư duy theo miền nghiệp vụ, không theo cơ sở dữ liệu . . . 7
2.3. Strategic DDD — Bounded Context & Context Map . . . 7
2.3.1. Bounded Context — ranh giới ngôn ngữ . . . 7
2.3.2. Xác định Bounded Context từ yêu cầu nghiệp vụ . . . 9
2.3.3. Phân rã hướng dịch vụ theo Erl — cách tiếp cận step-by-step . . . 10
2.4. Case study: KBM . . . 13
2.4.1. Phân tích domain . . . 13
2.4.2. Bảy Bounded Context core . . . 14
2.5. Shared Library và Shared Kernel — chia sẻ hay không chia sẻ? . . . 16
2.5.1. Vấn đề thực tế . . . 16
2.5.2. Hướng cải thiện . . . 17
2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service . . . 18
2.7. Assemblage Process — quy trình thiết kế service architecture . . . 20
2.8. Tổng kết . . . 20
2.9. Đọc thêm . . . 21
Tài liệu tham khảo . . . 22

CHƯƠNG 2

Phân tích hướng domain — DDD & Bounded Contexts

> "Use the model as the backbone of a language. Commit the team to exercising that language relentlessly in all communication within the team and in the code."
> — Eric Evans, *Domain-Driven Design*

MỤC TIÊU HỌC TẬP

- Hiểu Conway's Law và tác động của cấu trúc tổ chức lên kiến trúc phần mềm
- Nắm vững các khái niệm DDD cốt lõi: Entity, Value Object, Aggregate, Repository
- Áp dụng Strategic DDD: Bounded Context, Context Map, Ubiquitous Language
- Thực hành xác định ranh giới dịch vụ từ domain — kỹ năng quan trọng nhất khi thiết kế microservices
- So sánh hai phương pháp phân tách: DDD (heuristic-based) vs Erl Service-Oriented Analysis (step-by-step)
- Phân tích case study KBM: 7 Bounded Context core cùng 2 supporting context; phân biệt Shared Library với Shared Kernel khi quyết định chia sẻ mã nguồn

2.1. Conway's Law — tổ chức quyết định kiến trúc

Một trong những quy luật nền tảng ảnh hưởng đến mọi quyết định kiến trúc — đặc biệt quan trọng khi áp dụng Domain-Driven Design — là: **kiến trúc phần mềm phản ánh cấu trúc tổ chức phát triển nó**.

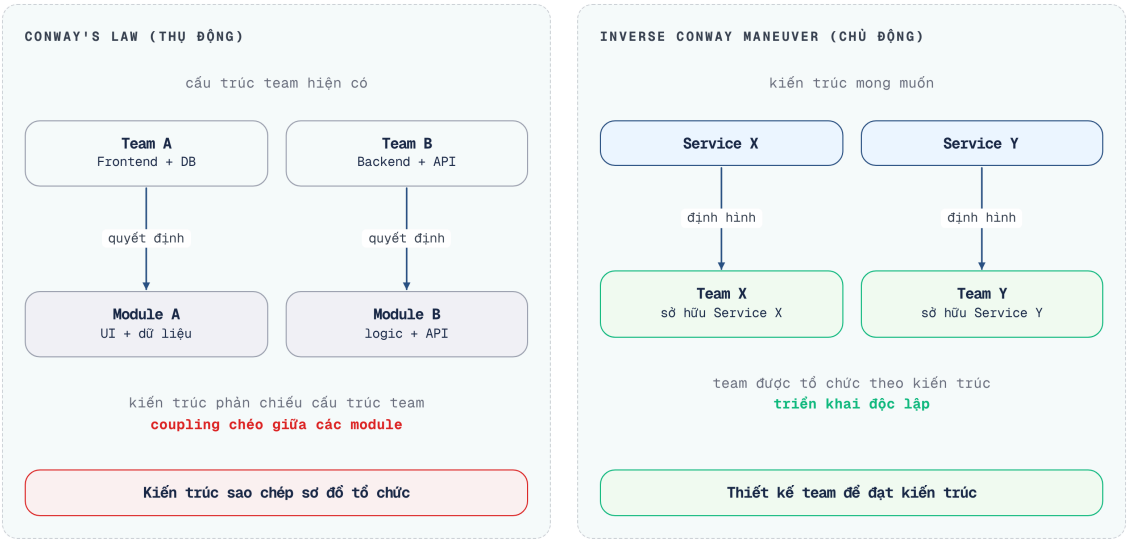
Quy luật Conway. Năm 1968, Melvin Conway quan sát rằng:

> **Conway's Law**
>
> "Organizations which design systems ... are constrained to produce designs which are copies of the communication structures of these organizations."
>
> — *Melvin Conway, 1968 (trích lại trong Richardson [2a, Ch.2])*

Nói cách khác: một tổ chức có 3 team sẽ có *xu hướng* tạo ra hệ thống xoay quanh 3 khối lớn (service hoặc module) — không phải ánh xạ 1:1 tuyệt đối, mà là ranh giới hệ thống phản chiếu ranh giới giao tiếp. Nếu hai nhóm phát triển giao tiếp với nhau tần suất cao, hai dịch vụ tương ứng cũng sẽ hình thành sự liên kết chặt chẽ (tight coupling). Đây không phải lý thuyết suông — nghiên cứu của MacCormack, Rusnak & Baldwin tại Harvard Business School (2008) đã cung cấp bằng chứng thực nghiệm hỗ trợ quy luật này trên các dự án mã nguồn mở.

Đối với lập trình viên và kiến trúc sư phần mềm, điều này có ý nghĩa thực tiễn rõ ràng: **không thể thiết kế microservices tốt nếu cấu trúc team không phù hợp**. Đây là lý do Amazon tái cấu trúc thành "two-pizza teams" *trước* khi phân rã hệ thống nguyên khối (monolithic system) (đã thảo luận ở Chương 1).

Inverse Conway Maneuver. Thay vì chấp nhận thụ động, nhiều tổ chức chủ động áp dụng Conway's Law theo hướng ngược lại: **thiết kế cấu trúc team để đạt được kiến trúc phần mềm mong muốn**. Chiến lược này gọi là *Inverse Conway Maneuver*.



Hình 2.1: *Conway's Law (thụ động) vs Inverse Conway Maneuver (chủ động)*

Theo mô hình này, một nhóm phát triển phụ trách một hoặc một vài dịch vụ liên quan, chịu trách nhiệm cho toàn bộ vòng đời (lifecycle) của phần mềm — từ lúc viết mã nguồn đến khi vận hành

trên production. Đây là mô hình "you build it, you run it" mà Amazon và Netflix đã chứng minh hiệu quả.

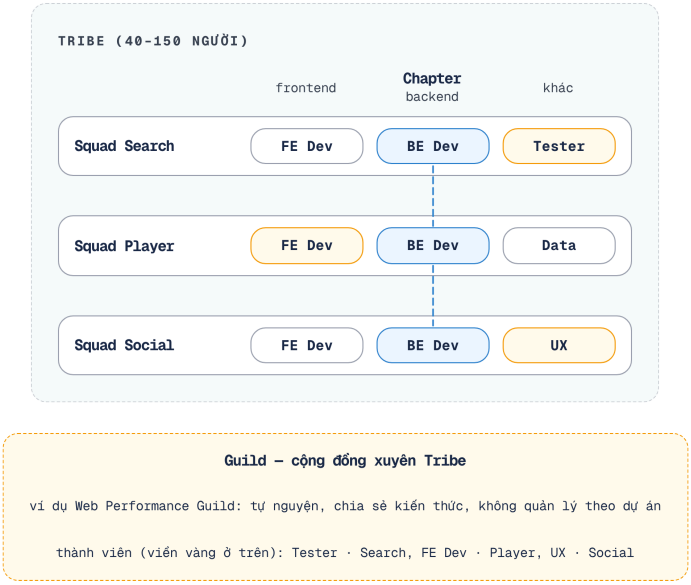
Team Topologies — Bốn kiểu team. Matthew Skelton và Manuel Pais đề xuất bốn kiểu team cơ bản cho tổ chức phần mềm hiện đại:

__Bảng 2.1:__ Bốn kiểu team theo Team Topologies

Kiểu team	Vai trò	Nếu KBM có nhiều nhóm (giả định)
Stream-aligned	Phát triển theo luồng nghiệp vụ, sở hữu trọn luồng	Team "Bài tập & Chấm thi", Team "Quản lý người dùng"
Platform	Cung cấp nền tảng nội bộ giúp các nhóm stream-aligned phát triển nhanh hơn	Team DevOps (CI/CD, Docker, monitoring)
Enabling	Hỗ trợ các team khác nâng cao kỹ năng	Team kiến trúc (DDD coaching, code review)
Complicated subsystem	Sở hữu thành phần phức tạp đòi hỏi chuyên môn sâu	Team "SQL Judge" (sandbox isolation, multi-DBMS)

Với KBM — một dự án nhỏ (1–2 lập trình viên chính, cộng sinh viên hỗ trợ theo đợt) — không thể áp dụng đầy đủ 4 mô hình nhóm. Tuy nhiên, nguyên lý cốt lõi vẫn được giữ nguyên: lập trình viên nào am hiểu sâu sắc về một miền nghiệp vụ (domain) sẽ tự nhiên đảm nhận trách nhiệm chính cho dịch vụ tương ứng. Khi nhóm mở rộng, cấu trúc này sẽ trở nên tường minh hơn.

Case study thực tế: Spotify Squad Model. Spotify (2012) là case study nổi tiếng nhất về tổ chức team cho microservices. Henrik Kniberg và Anders Ivarsson mô tả mô hình gồm bốn thành phần:



__Hình 2.2:__ Mô hình Squad/Tribe/Chapter/Guild của Spotify

Chi tiết về cấu trúc này và ý nghĩa của chúng đối với kiến trúc microservices được tóm tắt trong bảng sau.

__Bảng 2.2:__ Cấu trúc tổ chức Spotify — ý nghĩa cho microservices

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Squad (6–12 người)	Đơn vị tự trị, sở hữu trọn một tính năng	Tương ứng stream-aligned team, sở hữu một hoặc vài microservices

Cấu trúc	Mô tả	Ý nghĩa cho microservices
Tribe (thường ước lượng 40–150 người; dưới khoảng 100 người theo Kniberg & Ivarsson 2012)	Nhóm các squad cùng sứ mệnh (mission)	Đồng bộ mục tiêu mà không mất tính tự trị
Chapter	Nhóm người cùng năng lực chuyên môn xuyên các squad	Chia sẻ kinh nghiệm và chuẩn kỹ thuật chung (ví dụ: tất cả kỹ sư lập trình backend)
Guild	Cộng đồng quan tâm xuyên các tribe	Chia sẻ kiến thức (ví dụ: Web Performance Guild)

Bài học cho hệ thống nhỏ: Spotify áp dụng mô hình này khi đã có hơn 30 squad (Kniberg & Ivarsson, 2012) — quy mô đội KBM hiện nay rất nhỏ nên chưa cần mô hình squad/tribe. Nhưng nguyên tắc cốt lõi vẫn giữ nguyên giá trị: mỗi dịch vụ cần có người chịu trách nhiệm (owner) rõ ràng, và khi nhóm mở rộng, cần tổ chức nhân sự theo các Bounded Context (ranh giới nghiệp vụ độc lập — khái niệm trọng tâm sẽ được định nghĩa ở §2.3) thay vì theo technical layer.

Nguồn: Henrik Kniberg, Anders Ivarsson, "Scaling Agile @ Spotify," 2012. Lưu ý: Spotify đã tiến hóa/cải tiến mô hình này đáng kể từ năm 2012 — mô hình nguyên thủy chỉ là điểm khởi đầu, không phải là một khuôn mẫu cứng nhắc.

Team nhỏ và Conway's Law

Trong KBM, cùng một lập trình viên phát triển nhiều context (chẳng hạn SQL Practice và Academic Management). Điều này dẫn đến ranh giới giữa các service không rõ ràng — ví dụ: `spring.application.name` trùng nhau giữa `kbm-core` và `kbm-academic` (cả hai đều là `app`), shared database vẫn được duy trì vì tiện lợi. Richardson trong [2a] cho thấy việc định nghĩa rõ service ownership từ đầu giúp tránh sự phụ thuộc ngầm (implicit coupling). **Lộ trình chuyển đổi (Migration path)**: tách rõ naming, định nghĩa API contract giữa các service, và chuẩn bị cho việc mở rộng team.

2.2. DDD cơ bản — ngôn ngữ chung và các building blocks

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm đặt **domain** (miền nghiệp vụ) làm trung tâm. Eric Evans giới thiệu DDD năm 2003 trong cuốn sách cùng tên, và nó trở thành nền tảng cho việc phân tách microservices.

Đối với lập trình viên, DDD trả lời câu hỏi quan trọng nhất khi thiết kế microservices: **tách ở đâu?** Không phải tách theo layer kỹ thuật (UI team, DB team, API team) mà tách theo **ranh giới nghiệp vụ** — nơi mà sự thay đổi trong một domain không ảnh hưởng đến domain khác.

2.2.1. Ubiquitous Language — ngôn ngữ chung

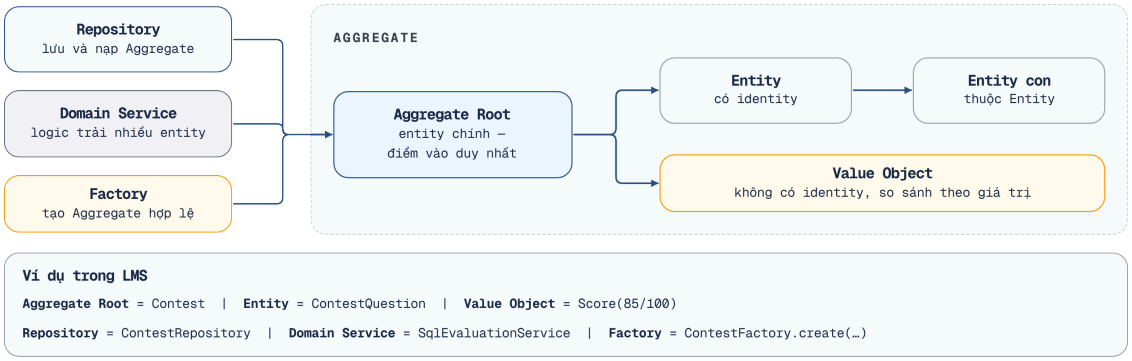
Bước đầu tiên và quan trọng nhất trong DDD là **xây dựng ngôn ngữ chung** (*Ubiquitous Language*) giữa lập trình viên và chuyên gia nghiệp vụ (domain expert).

Ubiquitous Language là tập hợp thuật ngữ mà *toàn bộ nhóm phát triển* — lập trình viên, kiểm thử viên, quản lý sản phẩm — sử dụng nhất quán trong mọi giao tiếp: email, các cuộc họp, mã nguồn, tài liệu. Nếu domain expert nói "bài nộp" (*submission*), code phải có class `Submission`, database có bảng `submissions`, API có endpoint `/submissions`. Không được có sự khác biệt.

Tại sao điều này quan trọng cho microservices? Vì khi hai nhóm phát triển bắt đầu dùng cùng một thuật ngữ nhưng với **ý nghĩa khác nhau**, đó là tín hiệu họ đang ở hai Bounded Context khác nhau; tách thành hai service triển khai riêng hay không còn tùy các lực đánh đổi ở §2.6.

2.2.2. Các building blocks

DDD định nghĩa một bộ building blocks để mô hình hóa domain:



Hình 2.3: Các building blocks trong DDD — Aggregate, Entity, Value Object, Repository

Entity — Đối tượng có danh tính (*identity*) duy nhất xuyên suốt vòng đời. Hai entity có cùng thuộc tính nhưng khác ID vẫn là hai entity khác nhau. Ví dụ trong KBM: `Student(id=1, name="Nguyen Van A")` và `Student(id=2, name="Nguyen Van A")` là hai sinh viên khác nhau dù trùng tên.

Value Object — Đối tượng được xác định bởi *giá trị*, không có identity riêng. Hai value object cùng giá trị là tương đương. Ví dụ: `Score(value=85, maxValue=100)` — không quan trọng "đây là điểm nào", chỉ quan trọng giá trị 85/100.

Aggregate — Cụm entity và value object được quản lý như một đơn vị nhất quán (*consistency boundary*). Mỗi aggregate có một **Aggregate Root** — entity duy nhất mà bên ngoài có thể tham chiếu. Trong LMS, Contest có thể là aggregate root bao gồm ContestQuestion, ContestParticipant — không một thành phần bên ngoài nào được phép *thay đổi dữ liệu* ContestQuestion mà không thông qua Aggregate Root là Contest. (Lưu ý: Có thể dùng SELECT trực tiếp để đọc dữ liệu, nhưng khi thao tác ghi (*Write*), mọi logic bắt buộc phải qua Root để tránh data corruption).

Aggregate = Transaction boundary

Trong microservices, aggregate chính là đơn vị transaction. Một transaction chỉ nên thao tác trên **một aggregate** duy nhất; khi quy trình nghiệp vụ chạm nhiều aggregate, hãy ưu tiên phối hợp qua domain event và chấp nhận eventual consistency. Khi nghiệp vụ trải qua *nhiều service* — mỗi service một database — transaction chung không còn khả thi: đó mới là lúc cần Saga pattern (Chương 6). Việc sử dụng `@Transactional` để cập nhật nhiều aggregate trong cùng một giao dịch cơ sở dữ liệu vẫn có thể chấp nhận được trong monolith, nhưng trong microservices, mở rộng một transaction xuyên hai service sẽ làm mất khả năng mở rộng (scale) độc lập và tạo nguy cơ khóa lẫn nhau giữa các dịch vụ.

Repository — Interface cho phép làm việc với aggregate như với một collection trong bộ nhớ. Lập trình viên làm việc với repository thay vì database trực tiếp. Trong Spring Boot: `@Repository interface ContestRepository extends JpaRepository<Contest, UUID>`.

Domain Service — Logic nghiệp vụ không thuộc về một entity hay value object cụ thể nào. Ví dụ: `SqlEvaluationService` so sánh kết quả SQL của sinh viên với đáp án — logic này không thuộc về `Submission` hay `Question` riêng lẻ.

2.2.3. DDD dành cho kỹ sư phần mềm: tư duy theo miền nghiệp vụ, không theo cơ sở dữ liệu

Sai lầm phổ biến nhất khi bắt đầu với DDD là **thiết kế entity từ bảng database**. DDD yêu cầu ngược lại: mô hình hóa domain trước, database sau.

Hai cách tiếp cận thiết kế phổ biến mang lại những kết quả hoàn toàn trái ngược nhau:

- **Database-first ×**: Bắt đầu từ bảng, cột và khóa ngoại (foreign key). Kết quả thường dẫn đến entity phản ánh nguyên vẹn cấu trúc bảng, dịch vụ trở thành các lớp bọc (wrapper) đơn thuần để thao tác dữ liệu (CRUD), và ranh giới nghiệp vụ giữa các thành phần bị mờ nhạt.
- **Domain-first ✓**: Bắt đầu từ quy trình nghiệp vụ và ngôn ngữ chuyên ngành (domain language). Kết quả là entity phản ánh chính xác các khái niệm nghiệp vụ, tạo ra những ranh giới tự nhiên và linh hoạt khi hệ thống cần mở rộng hoặc chia tách.

Trong thực tế, nhiều dự án (kể cả KBM) bắt đầu database-first. Ở nhiều hệ thống, khó khăn nghiêm trọng nhất của cách làm này là các bảng dữ liệu tập trung khổng lồ "God Table" (ví dụ minh họa: bảng `User` phình to thành 50 cột chứa hỗn hợp nhiều loại dữ liệu từ Auth, Billing đến Profile). Khi phân rã thành các microservice, các dịch vụ vẫn phải truy cập trực tiếp vào cùng một bảng `User` này, tạo ra sự phụ thuộc ngầm (coupling). Bước đầu tiên phải là *nghĩ lại* mô hình từ góc nhìn domain để phân rã (decompose) "God Table" đó ra.

Tư duy Database-first vs Domain-first

Có thể hình dung qua việc xây dựng Phòng Đào tạo. Với tư duy "Database-first", bước đầu tiên là mua các tủ hồ sơ, kệ sách, máy tính rồi sau đó mới nghĩ xem nhân viên sẽ làm việc thế nào với các trang thiết bị đó. Ngược lại, "Domain-first" yêu cầu vẽ ra quy trình trước: sinh viên nộp hồ sơ ở đâu, giáo vụ duyệt thế nào, trả kết quả ra sao, từ đó mới quyết định cần mua bao nhiêu tủ, sắp xếp bàn làm việc ra sao.

2.3. Strategic DDD — Bounded Context & Context Map

Nếu tactical DDD (Entity, Aggregate) giúp mô hình hóa *bên trong* một service, thì strategic DDD giúp trả lời câu hỏi lớn hơn: **hệ thống cần bao nhiêu service, và ranh giới ở đâu?** Để làm được điều này, DDD phân biệt rõ:

- **Problem Space (Không gian vấn đề)**: Chia domain lớn thành các Subdomain (subdomain cốt lõi, hỗ trợ hoặc chung).
- **Solution Space (Không gian giải pháp)**: Ánh xạ các Subdomain này thành các Bounded Context để cài đặt bằng code (lý tưởng là 1 Subdomain = 1 Bounded Context).

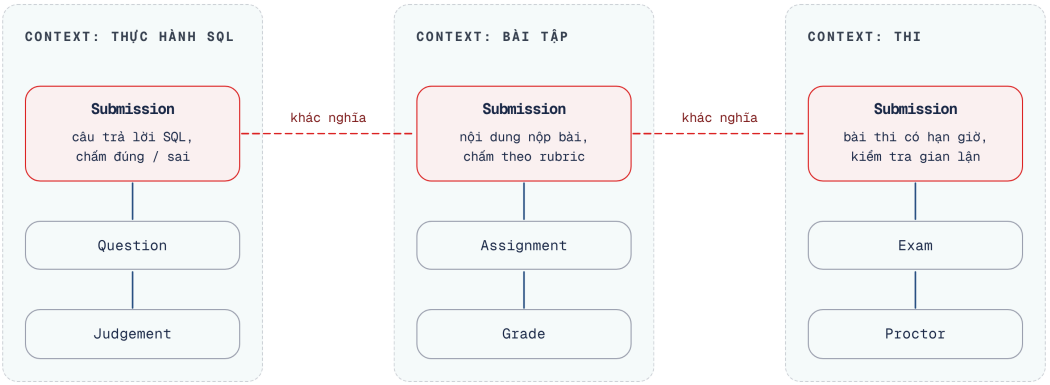
2.3.1. Bounded Context — ranh giới ngôn ngữ

Bounded Context là khái niệm quan trọng nhất trong DDD cho microservices. Một Bounded Context là phạm vi trong đó một mô hình domain cụ thể có hiệu lực — và **ngôn ngữ có ý nghĩa nhất quán**. Khái niệm này có thể được ví như một Khoa trong trường đại học quản lý độc lập sinh viên và điểm số của khoa đó, không cho phép khoa khác can thiệp trực tiếp dữ liệu.

Ví dụ thực tế: trong KBM, thuật ngữ "bài nộp" (*submission*) có ý nghĩa khác nhau tùy ngữ cảnh:

- Trong context **Thực hành SQL**: submission = câu trả lời SQL của sinh viên, cần chấm đúng/sai
- Trong context **Bài tập**: submission = file/nội dung nộp bài, cần chấm điểm theo rubric
- Trong context **Thi**: submission = bài thi với giới hạn thời gian, cần kiểm tra gian lận

Cùng một từ, ba ý nghĩa khác nhau → ba Bounded Context tiềm năng. Đây là heuristic hiệu quả nhất để xác định ranh giới: **khi cùng một thuật ngữ mang ý nghĩa khác nhau, chúng ta đang ở ranh giới giữa hai context**.



Hình 2.4: *Cùng thuật ngữ "Submission" mang ý nghĩa khác nhau trong ba Bounded Context*

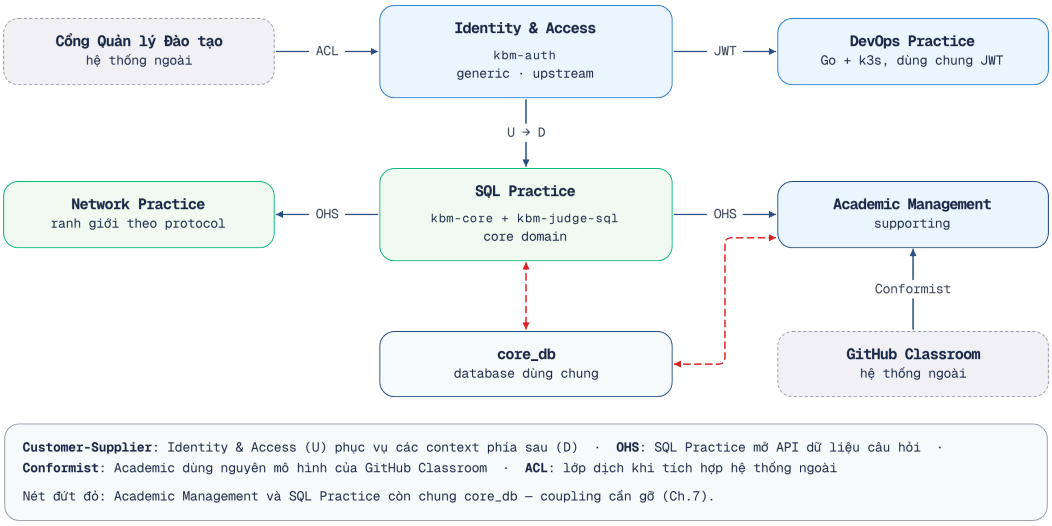
Context Map — Bản đồ quan hệ giữa các context. Khi đã xác định các Bounded Context, bước tiếp theo là vẽ **Context Map** — sơ đồ quan hệ giữa chúng. Context Map không chỉ cho thấy context nào giao tiếp với nhau, mà còn cho thấy **kiểu quan hệ**.

Các kiểu quan hệ phổ biến:

Bảng 2.3: *Các kiểu quan hệ giữa các Bounded Context*

Quan hệ	Mô tả	Ví dụ trong KBM
Partnership	Hai context phát triển đồng bộ, hợp tác chặt chẽ	Chưa có trong KBM (đội quá nhỏ); sẽ xuất hiện nếu hai nhóm cùng phát triển Core và Academic
Shared Kernel	Hai context cùng sở hữu và cùng quản trị một phần mô hình *miền*	Nhóm khái niệm miền dùng chung (vd: value object mã sinh viên, thang điểm). Phân biệt: thư viện kỹ thuật (base entities, JWT, exceptions) là Shared *Library*, không phải Shared Kernel
Customer-Supplier	Nhóm Upstream (U) ưu tiên phát triển theo yêu cầu nhóm downstream (D)	Identity & Access (U) → SQL Practice (D)
Conformist	(D) chấp nhận nguyên mô hình của (U) vì (U) không có động cơ đáp ứng — khác Customer-Supplier ở chỗ không thương lượng được	Academic Management dùng nguyên mô hình lớp/bài tập của GitHub Classroom: không thể yêu cầu bên ngoài đổi contract
Anti-Corruption Layer	(D) đặt lớp dịch trung gian ở biên để chuyển đổi mô hình của (U) sang mô hình nội bộ, ngăn mô hình ngoại lai xâm nhập domain	`kbm-auth` dựng lớp dịch khi tích hợp Cổng Quản lý Đào tạo (hệ thống ngoài) — model bên ngoài không xâm nhập domain KBM (Chương 4, Chương 10)
Open Host Service	(U) mở giao thức/API chuẩn hóa, có tài liệu rõ ràng để phục vụ đồng thời nhiều (D) tích hợp	`kbm-auth` cung cấp API xác thực/JWT chuẩn cho toàn hệ thống; hoặc `kbm-core` mở API dữ liệu câu hỏi cho các context khác khi tách DB (Ch.7)
Published Language	Ngôn ngữ trao đổi được chuẩn hóa (JSON, Protobuf)	Kafka messages với schema chuẩn

Áp dụng các mẫu quan hệ này vào KBM, chúng ta có Context Map của hệ thống ở Hình 2.5.



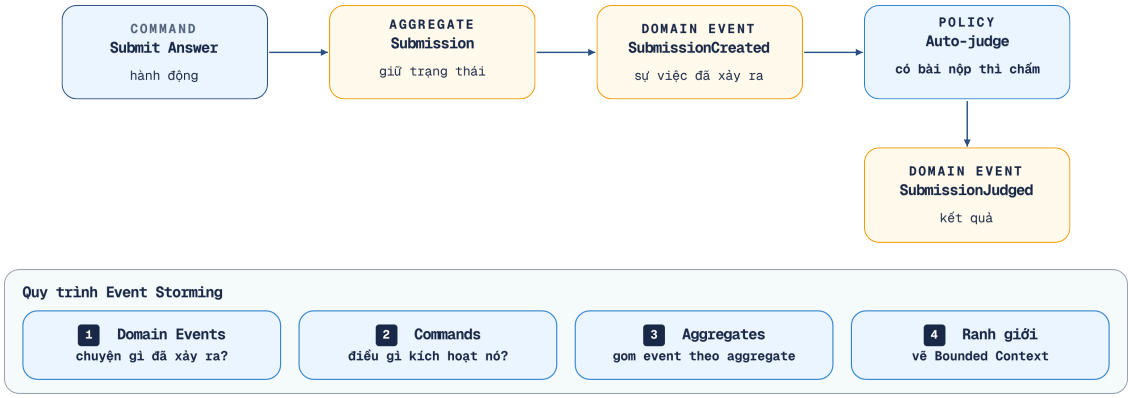
Hình 2.5: *Context Map của KBM — quan hệ giữa các Bounded Context*

2.3.2. Xác định Bounded Context từ yêu cầu nghiệp vụ

Làm thế nào để "tìm" Bounded Context trong một hệ thống? Không có công thức chính xác — đây là kỹ năng cần luyện tập. Tuy nhiên, có một số *heuristic* hữu ích (heuristic là quy tắc kinh nghiệm — lối suy luận thực dụng giúp ra quyết định nhanh khi không có công thức chính xác):

1. **Heuristic ngôn ngữ** — Khi cùng một thuật ngữ mang ý nghĩa khác nhau cho các nhóm người khác nhau, đó là ranh giới context. Ngược lại, khi hai thuật ngữ khác nhau chỉ cùng một thứ (ví dụ: "ref-des" và "component instance" trong câu chuyện PCB của Eric Evans), chúng thuộc cùng context.
2. **Heuristic thay đổi** — Tự hỏi: "Khi yêu cầu nghiệp vụ X thay đổi, những đoạn mã nguồn nào cần sửa?" Nếu thay đổi luôn ảnh hưởng đến cùng một nhóm tệp tin (files), chúng thuộc cùng context. Nếu thay đổi lan sang nhóm khác, đó là tín hiệu coupling cần xem xét.
3. **Heuristic team** — Bộ phận nào sẽ đảm nhận phát triển và bảo trì phần này? Nếu do cùng một nhóm phụ trách, các thành phần này *có thể* thuộc cùng một ngữ cảnh. Nếu do các nhóm khác nhau đảm nhận, cần *cân nhắc* việc phân tách ngữ cảnh (Conway's Law).
4. **Heuristic data** — Dữ liệu nào "thuộc về" nhau? Những entity nào phải cùng giữ một bất biến nghiệp vụ trong một transaction? Đó là ứng viên cho cùng một aggregate — chỉ thường được truy vấn cùng nhau thì chưa đủ để kết luận. Entity nào chỉ được tham chiếu qua ID? Đó có thể thuộc context khác.

Event Storming — Phương pháp khám phá domain. Event Storming là kỹ thuật workshop do Alberto Brandolini phát triển, giúp cả team cùng khám phá domain thông qua *domain events* — những sự kiện đã xảy ra trong hệ thống.



Hình 2.6: *Event Storming — luồng từ Command đến Domain Event*

Quy trình cơ bản:

1. **Liệt kê domain events** — giấy ghi chú (post-it) màu cam: "Submission Created", "SQL Executed", "Score Calculated"
2. **Tìm commands** — giấy ghi chú màu xanh: "Submit Answer", "Start Exam", "Create Assignment"
3. **Nhóm event theo aggregate** — giấy ghi chú màu vàng: những event nào cùng xảy ra trên một entity?
4. **Vẽ ranh giới** — nhóm các aggregate có liên quan chặt chẽ → Bounded Context

Kết quả Event Storming tự nhiên dẫn đến các Bounded Context. Nhóm event/aggregate nào tương tác nhiều nội bộ nhưng ít tương tác với nhóm khác → đó là một context.

2.3.3. Phân rã hướng dịch vụ theo Erl — cách tiếp cận step-by-step

DDD và Event Storming là phương pháp *khám phá* — bắt đầu từ ngôn ngữ domain, dần dần tìm ra ranh giới. Cách tiếp cận này mạnh nhưng đòi hỏi kinh nghiệm: các lập trình viên thiếu kinh nghiệm thường gặp khó khăn trong việc xác định điểm dừng hoặc tính đúng đắn của ranh giới.

Thomas Erl trong *Service-Oriented Architecture: Analysis and Design for Services and Microservices* [1] đề xuất một phương pháp bổ trợ: **Service-Oriented Analysis** — quy trình *prescriptive*, gồm các bước cụ thể từ yêu cầu nghiệp vụ đến danh mục dịch vụ. Giáo trình tóm lược tinh thần phương pháp này thành năm bước dễ dạy và dễ học, mỗi bước có input và output rõ ràng.

Năm bước phân rã dịch vụ (giáo trình tổng hợp theo tinh thần Erl [1]):

1. **Xác định automation candidates (ứng viên tự động hóa)** — Phân tích quy trình nghiệp vụ (business process) và xác định những bước nào *nên* được tự động hóa bằng phần mềm. Erl gọi đây là các *service candidate* — chưa phải service hoàn chỉnh, mà là ứng viên.

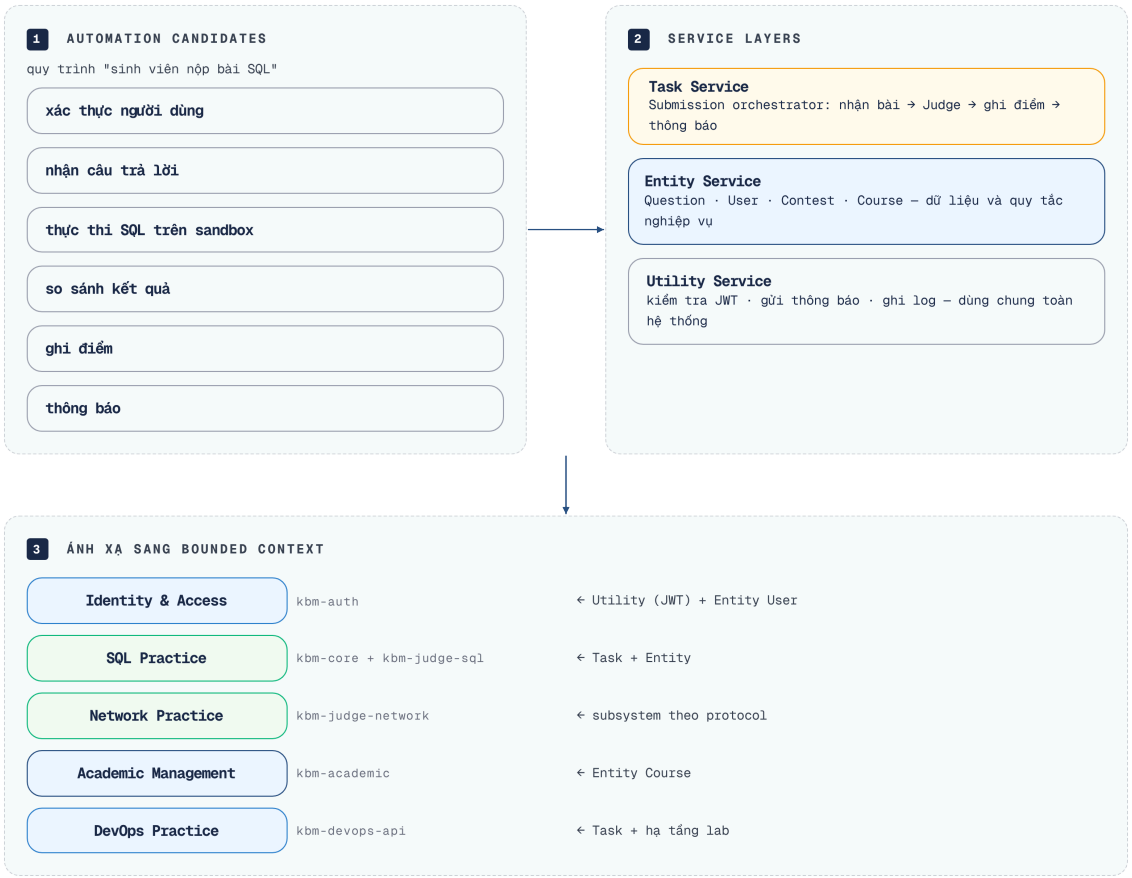
   Ví dụ KBM: quy trình "Sinh viên nộp bài SQL" có các bước: (1) xác thực người dùng, (2) nhận câu trả lời, (3) thực thi SQL trên sandbox, (4) so sánh kết quả, (5) ghi điểm, (6) thông báo. Mỗi bước là một automation candidate.
2. **Phân loại service layers** — Erl định nghĩa ba loại service:
   - **Task Service**: Chịu trách nhiệm điều phối quy trình nghiệp vụ (workflow) bằng cách gọi các service khác. Ví dụ trong KBM: Submission orchestrator thực hiện chuỗi xử lý từ nhận bài, gọi dịch vụ chấm điểm (Judge), ghi điểm cho đến thông báo.

- **Entity Service**: Chuyên quản lý dữ liệu nghiệp vụ cùng các thao tác CRUD và quy tắc nghiệp vụ (business rules). Ví dụ: Question Service, User Service, Contest Service.
- **Utility Service**: Cung cấp các chức năng kỹ thuật được dùng chung trên toàn hệ thống. Ví dụ: Dịch vụ kiểm tra JWT (validation), gửi thông báo (notification), hoặc ghi nhật ký hệ thống (logging).

Phân loại này giúp tránh sai lầm phổ biến: tạo ra các dịch vụ chỉ đơn thuần thực hiện CRUD (entity service không có business logic) hoặc gom toàn bộ logic vào một "god service" (thiếu task/utility decomposition).

3. **Tạo service inventory blueprint** — Vẽ bản đồ toàn bộ các service dự kiến, mỗi dịch vụ với các thao tác (operation) chính. Tương tự Context Map trong DDD, nhưng tập trung hơn vào các năng lực (*capabilities*) thay vì *language boundaries* (ranh giới ngôn ngữ).
4. **Xác định service boundaries + composition** — Kiểm tra từng service candidate: nó có thể hoạt động **độc lập** không? Nếu dịch vụ A *luôn* cần gọi dịch vụ B trước khi có thể xử lý bất kỳ yêu cầu (request) nào → cân nhắc việc gộp chung. Nếu dịch vụ A vẫn có thể hoạt động độc lập ngay cả khi dịch vụ B gặp sự cố (ngừng hoạt động) → ranh giới tốt.
5. **Kiểm tra theo 8 nguyên lý SOA** — Mỗi service phải thỏa mãn 8 nguyên lý hướng dịch vụ đã thảo luận ở Bảng 1.2 (Chương 1): Standardized Contract, Loose Coupling, Abstraction, Reusability, Autonomy, Statelessness, Discoverability, Composability. Đây là "checklist kiểm tra chất lượng" cho service design.

Áp dụng năm bước trên cho KBM cho kết quả như Hình 2.7, đi từ các automation candidate đến các lớp dịch vụ (service layer).



Hình 2.7: *Áp dụng Erl's Service-Oriented Analysis cho KBM — từ automation candidate đến service layer*

Kết quả ở bước minh họa này cho ra 5 nhóm dịch vụ tiêu biểu — trùng khớp với năm Bounded Context tương ứng trong phân tích DDD (§2.4): Identity & Access ≈ Utility Auth, SQL Practice ≈ Entity Question + Task Orchestrator, Network Practice ≈ subsystem giao thức chuyên biệt (khái niệm *complicated-subsystem* mượn từ Team Topologies — nằm ngoài taxonomy Erl), Academic Management ≈ Entity Course, và DevOps Practice ≈ Task Service + Infrastructure Automation. Hai Bounded Context core còn lại trong tổng số bảy context của KBM — Code Practice (tích hợp DOMjudge thay cho bộ chấm MVC thế hệ đầu) và Examination & Proctoring (hiện điều phối cùng `kbm-core`) — được trình bày đầy đủ ở §2.4.

So sánh: Erl vs DDD

Bảng 2.4 đặt hai phương pháp cạnh nhau theo từng tiêu chí.

Bảng 2.4: *Hai phương pháp phân tách service — Erl Service-Oriented Analysis vs DDD*

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Điểm bắt đầu	Quy trình nghiệp vụ (business process)	Ngôn ngữ domain (Ubiquitous Language)
Phong cách	Theo quy trình — 5 bước rõ ràng, input/output mỗi bước	Khám phá — heuristics, Event Storming workshop
Tiêu chí tách	Service layers (task/entity/utility) + 8 nguyên lý SOA	Ranh giới ngôn ngữ + aggregate consistency
Output	Service inventory blueprint	Context Map + Bounded Context diagram
Thế mạnh	Có hệ thống, dễ dạy, hợp nhóm mới	Sâu về domain, phát hiện ranh giới ẩn

Tiêu chí	Erl Service-Oriented Analysis	DDD Bounded Context
Hạn chế	Có thể tạo ranh giới quá kỹ thuật (thiếu domain insight)	Cần kinh nghiệm, kết quả phụ thuộc người phân tích
Phù hợp khi	Team ít kinh nghiệm DDD, cần quy trình chuẩn	Team hiểu domain sâu, cần phát hiện complexity ẩn

Dù tiếp cận theo cách nào, mục tiêu cuối cùng vẫn là tìm ra ranh giới hợp lý nhất cho hệ thống. Thay vì coi đây là hai trường phái đối lập, hãy xem chúng là các công cụ hỗ trợ lẫn nhau.

Bổ trợ, không thay thế

Erl cung cấp *quy trình* (process) — đặc biệt hữu ích khi chúng ta chưa biết bắt đầu từ đâu. DDD cung cấp *insight* — phát hiện những ranh giới mà quy trình cơ học bỏ sót. Trong thực tế: dùng Erl để tạo bản phác thảo ban đầu (service inventory), sau đó dùng DDD (Event Storming, ngôn ngữ chung) để *tinh chỉnh* ranh giới. Richardson's Assemblage Process (§2.7) kết hợp cả hai hướng.

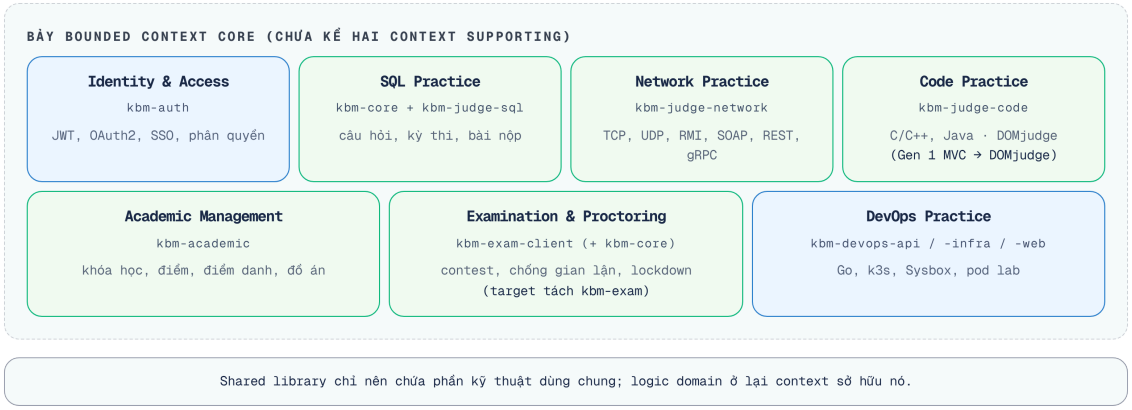
2.4. Case study: KBM

Trở lại với bài toán hệ thống LMS của trường đại học, chúng ta không thể giao toàn bộ việc quản lý điểm số, bài tập và kỳ thi cho cùng một bộ phận. Việc chia nhỏ thành các Bounded Context giúp định hình rõ quyền hạn và trách nhiệm của từng phòng ban.

2.4.1. Phân tích domain

KBM phục vụ các hoạt động giảng dạy, thực hành, đánh giá và vận hành lab thực hành. Từ góc nhìn DDD, chúng ta phân tách Problem Space thành các Subdomain: **SQL Practice** là Core Domain (cốt lõi tạo ra giá trị); **Identity & Access** là Generic Subdomain (đã có chuẩn chung ngoài thị trường); các mảng **Network Practice**, **Code Practice**, **Academic Management**, **Examination & Proctoring**, **DevOps Practice** và **Communication** (thông báo, email) là các Supporting Subdomain (cần thiết để hệ thống vận hành trọn vẹn nhưng không tạo lợi thế cạnh tranh cốt lõi). Còn các service như Gateway (`kbm-gateway`), Registry (`kbm-registry`) và thư viện dùng chung (`kbm-shared`) thuộc lớp **Platform / Infrastructure** (hạ tầng kỹ thuật), cần phân biệt rõ chúng với Subdomain nghiệp vụ.

Bước tiếp theo là chuyển sang *không gian giải pháp*: bảy Subdomain trực tiếp phục vụ bài toán giảng dạy và thi cử được hiện thực hóa thành **bảy Bounded Context core**, kết hợp với **hai supporting context** lo phần nền tảng dùng chung cho cả hệ thống (**Communication** — `kbm-notification`, và **Platform / Infrastructure** — `kbm-gateway`, `kbm-registry`, `kbm-shared`), tạo thành chín context trong bản đồ kiến trúc KBM. Cần đọc cặp "core / supporting" ở đây đúng tầng: nó nói về *vai trò kiến trúc* của context (nghiệp vụ đối lập với nền tảng dùng chung), không trùng với `Core Domain` hay `Supporting Subdomain` ở không gian bài toán vừa nhắc ở trên. Theo Evans, một hệ thống có thể có nhiều context core nhưng chỉ nên có *một* Core Domain — với KBM, đó là SQL Practice.



Hình 2.8: Bảy Bounded Context core của KBM

2.4.2. Bảy Bounded Context core

1. Identity & Access (`kbm-auth`)

Domain rõ ràng nhất — quản lý danh tính người dùng, xác thực, phân quyền. Entities: `User`, `Role`, `Token`. Đây là context upstream — mọi context khác đều phụ thuộc vào kết quả authentication.

Trong KBM, context này được tách thành service riêng sớm nhất (`kbm-auth`), vì ranh giới rất rõ: thay đổi cơ chế xác thực (tài khoản nội bộ, Google/Microsoft OAuth2, email verification, SSO từ hệ thống đào tạo) không ảnh hưởng logic bài tập hay chấm thi.

2. SQL Practice (`kbm-core` + `kbm-judge-sql` — *Core Domain*)

Đây là **Core Domain** ban đầu — phần tạo ra giá trị chính cho hệ thống SQL Judge. Entities: `Question`, `Submission`, `Contest`, `UserContest`, `Statistic`. Toàn bộ quy trình từ "sinh viên xem câu hỏi → viết SQL → nộp bài → nhận kết quả" nằm trong context này.

Đây là context lớn nhất và phức tạp nhất. Bên trong nó có kiến trúc chấm SQL gồm một service điều phối `kbm-judge-sql` và các worker `kbm-sandbox-*` theo từng DBMS (`kbm-sandbox-mysql`, `kbm-sandbox-postgres`, `kbm-sandbox-sqlserver`, ...). Đây mới là nơi cần phân tích kỹ về các vấn đề định tuyến (routing), khả năng phục hồi (resilience), idempotency và quyền sở hữu (ownership) giữa thành phần điều phối (coordinator) và thành phần xử lý (worker); không nên nhầm với Network Practice.

3. Network Practice (`kbm-judge-network`)

Domain chuyên biệt cho bài thực hành lập trình mạng Java: TCP, UDP, RMI, SOAP, REST và gRPC. Khác với SQL Practice, sinh viên viết client trên IDE cá nhân và kết nối trực tiếp đến judge server. Context này minh họa ranh giới domain đến từ **protocol semantics**, không chỉ từ entity/database.

Network Practice được thiết kế tương đối độc lập với các service còn lại: nó phục vụ mục tiêu đánh giá giao tiếp qua mạng, có protocol contract riêng và không cần chia sẻ pipeline chấm SQL. Trong sách, context này chủ yếu minh họa **technology diversity** và boundary theo protocol, không phải một khoảng trống kiến trúc cần sửa như SQL Judge.

4. Academic Management (`kbm-academic`)

Quản lý khóa học, bài tập dạng rubric, đăng ký môn, điểm số, điểm danh, MCQ daily, syllabus/CLO, GitHub Classroom và đồ án tốt nghiệp. Entities: `Course`, `Assignment`, `Enrollment`, `Grade`, `AttendanceSession`, `Thesis`.

Context này được tách ra từ Core Service — nhưng *vẫn sử dụng chung cơ sở dữ liệu* (`core_db`). Đây là trade-off quan trọng mà chúng ta sẽ phân tích trong phần tiếp theo.

5. DevOps Practice (`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`)

Domain mới phục vụ bài thực hành Docker/Kubernetes: tạo lab environment cô lập, terminal web, validator script và progress tracking. Context này dùng Go, k3s và Sysbox, sử dụng stack công nghệ khác với LMS chính. Đây là ví dụ tốt cho polyglot microservices: dùng công nghệ khác khi domain và hạ tầng yêu cầu, nhưng vẫn chia sẻ danh tính (identity) qua JWT của KBM.

6. Code Practice (`kbm-judge-code`)

Domain chấm bài lập trình thuật toán đa ngôn ngữ (C/C++, Java, Python...) theo mô hình ICPC: sinh viên nộp mã nguồn, hệ thống biên dịch, chạy với bộ test, đối chiếu đầu ra (output) và trả về kết quả (Verdict). Trong giai đoạn đầu, hệ thống sử dụng một bộ chấm điểm *tự phát triển nội bộ* (chỉ C/C++ và Java, không sandbox); về sau được thay bằng `kbm-judge-code` tích hợp **DOMjudge** (hệ thống chấm bài trực tuyến mã nguồn mở chuẩn thi đấu). Context này tách khỏi SQL Practice vì *mô hình chấm khác hẳn*: SQL so sánh result set trên CSDL thật, còn Code Practice biên dịch–chạy–so sánh kết quả đầu ra (output) trong sandbox cô lập. Đây cũng là minh họa kinh điển cho quyết định **"xây hay mua" (build vs buy)** (sẽ phân tích ở Chương 4 và Chương 10).

7. Examination & Proctoring (*logic Contest/Anti-cheat hiện trong* `kbm-core`*, mục tiêu tách* `kbm-exam`*;* `kbm-exam-client`)

Domain thi cử và giám sát: vòng đời Contest (Scheduled → Open → Closed), bảng xếp hạng thời gian thực, chống gian lận (chuyển thẻ trình duyệt (tab), sao chép-dán (copy-paste), đổi IP/User-Agent) và đặc biệt là kỳ thi **trên phòng máy** qua ứng dụng desktop `kbm-exam-client` (lockdown toàn màn hình, whitelist IP phòng thi, giám sát từ phía máy khách (client-side proctoring)). Về mặt nghiệp vụ đây là một Bounded Context riêng, dù *hiện tại* phần điều phối thi vẫn nằm chung trong `kbm-core` (một biểu hiện của siêu dịch vụ ôm đồm — god-service) — mục tiêu chuyển đổi (migration target) hệ thống là phân tách thành `kbm-exam`. Context này minh họa nguyên tắc **một backend phục vụ nhiều loại client** (web luyện tập + desktop thi) và **phòng thủ chiều sâu (defense in depth)** cho chống gian lận (Ch.9).

Từ Bounded Context đến service

Bảng 2.5 ánh xạ bảy Bounded Context core sang các service triển khai, kèm lý do tách của từng context.

Bảng 2.5: *Ánh xạ Bounded Context → service trong KBM*

Bounded Context	Service	Lý do tách
Identity & Access	`kbm-auth`	Ranh giới rõ ràng, thay đổi auth ≠ thay đổi logic học tập
SQL Practice	`kbm-core`, `kbm-judge-sql`, sandbox DB services	Core Domain, cần sandbox isolation và chấm tự động
Network Practice	`kbm-judge-network`	Protocol đa dạng, state/routing khác REST API thông thường
Code Practice	`kbm-judge-code`	Mô hình chấm compile–run–compare khác SQL; tích hợp DOMjudge ("xây hay mua" — build vs buy)
Academic Management	`kbm-academic`	Domain học vụ, grade scheme, attendance, thesis
Examination & Proctoring	`kbm-exam-client` (+ logic trong `kbm-core`, target `kbm-exam`)	Thi cử, lockdown phòng máy, anti-cheat; nhiều loại client một backend
DevOps Practice	`kbm-devops-api`, `kbm-devops-infra`, `kbm-devops-web`	Domain lab infrastructure, Go/k3s/Sysbox, validator script

Khi đối chiếu bảng phân rã này, một điểm quan trọng cần lưu ý là mối quan hệ giữa ranh giới nghiệp vụ và cấu trúc triển khai vật lý.

Bounded Context ≠ service

Bounded Context là ranh giới logic. Một ngữ cảnh có thể được hiện thực hóa (implement) bởi nhiều dịch vụ (như SQL Practice = Core + Judge + sandbox DB services). Ngược lại, một kho mã nguồn (repository) cũng có thể tạo ra nhiều tệp thực thi (binary) phục vụ cùng context (như DevOps Practice = API + router + grader trong một codebase Go) — miễn là ranh giới domain rõ ràng.

2.5. Shared Library và Shared Kernel — chia sẻ hay không chia sẻ?

Trong môi trường đại học, có những thông tin bắt buộc phải dùng chung như mã sinh viên hay quy chế đào tạo cốt lõi. Tương tự, **Shared Kernel** là phần *mô hình miền* mà nhiều Bounded Context buộc phải đồng thuận chia sẻ, dù điều này làm giảm tính độc lập — khác với **Shared Library**, nơi chỉ gom mã hạ tầng kỹ thuật dùng chung.

2.5.1. Vấn đề thực tế

Trong KBM, tồn tại một **shared library** chứa 23 tệp Java: các lớp thực thể cơ sở (base entity), DTO, tiện ích JWT, xử lý ngoại lệ và tiện ích kiểm tra hợp lệ (validation). Nhiều service Java phụ thuộc vào library này.

Đúng tên gọi, `kbm-shared` là một **Shared Library**: nó gom mã nguồn hạ tầng dùng chung cho nhiều dịch vụ. Kiểu chia sẻ này giảm trùng lặp nhưng sinh ra phụ thuộc — mọi thay đổi trong thư viện đều *có thể* ảnh hưởng đến tất cả dịch vụ đang dùng nó.

Shared Kernel trong DDD là khái niệm khác hẳn. Thuật ngữ này chỉ dành cho phần **Domain Model** mà nhiều Bounded Context cố ý đồng sở hữu và cùng quản trị — chẳng hạn value object `StudentId` hay thang điểm `GradingScale` mà cả Academic Management lẫn Examination đều phải tuân theo. Vì vậy KBM chấp nhận Shared Library cho hạ tầng, nhưng hạn chế tối đa Shared Kernel trong domain.

Cấu trúc thư mục hiện tại của `kbm-shared`:

```
kbm-shared/
├── common/          BaseEntity, BaseMapper, BaseRequest, BaseResponse, BaseService
├── config/          AuditConfig, CorsConfig, TimeZoneConfig
├── enumerate/       AnswerStatus, ErrorCode, TypeQuestion
├── exception/       GlobalExceptionHandler
├── securityConfig/  JwtConfig, UserDetailCustom
└── utils/           CompareUtil, JwtUtil, SqlUtil, TableDependencyResolver
```

Phân tích: cái gì nên shared, cái gì không?

Bảng 2.6 phân loại nội dung của `kbm-shared` theo câu hỏi có nên dùng chung hay không.

__Bảng 2.6:__ Phân tích shared library — cái gì nên dùng chung, cái gì không

Loại	Ví dụ	Nên dùng chung?	Lý do
Cross-cutting concerns	JWT validation, exception handling, CORS config	✓ Có	Nhất quán hành vi giữa các dịch vụ, ít thay đổi

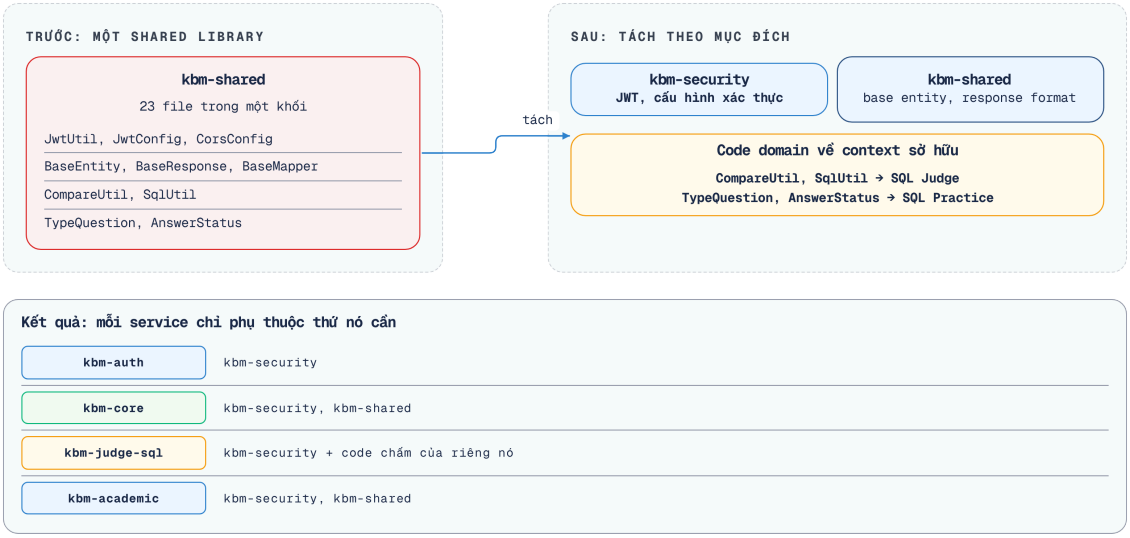
Loại	Ví dụ	Nên dùng chung?	Lý do
Base abstractions	BaseEntity, BaseMapper, BaseResponse	! Cân nhắc	Mang lại sự tiện lợi nhưng tạo ra sự phụ thuộc (coupling) cứng nhắc vào các quy ước chung (convention)
Domain-specific logic	`CompareUtil` (SQL comparison), `SqlUtil`	× Không	Chỉ Judge context cần — không phải shared concern
Domain enums	`TypeQuestion`, `AnswerStatus`	× Không	Thuộc về SQL Practice context, không universal

2.5.2. Hướng cải thiện

Nếu team phát triển lớn hơn và cần giảm coupling, shared library có thể được tách thành:

1. `kbm-security` — JWT, cấu hình xác thực (mọi dịch vụ đều cần)
2. `kbm-shared` — Base entities, response format (convention)
3. **Domain-specific code** → chuyển vào service tương ứng (thuộc SQL Practice, Code Practice, v.v.)

Sơ đồ dưới đây minh họa hướng tách này — mục tiêu là giảm phụ thuộc không cần thiết giữa các module:



Hình 2.9: *Hướng tách shared library — từ một khối sang nhiều module*

Chia sẻ mã nguồn là thỏa thuận có chủ đích

Dù là Shared Library hay Shared Kernel, việc chia sẻ đều phải là *thỏa thuận có chủ đích*, không phải sự tiện lợi. Mỗi file trong thư viện dùng chung phải trả lời được: "Thay đổi file này ảnh hưởng tới những service nào, và các service đó có *buộc phải* cập nhật cùng lúc không?" File chỉ phục vụ nghiệp vụ của một context thì thuộc về context đó; mã hạ tầng ổn định có thể dùng chung, miễn các service nâng phiên bản độc lập được.

Hai loại chia sẻ có tiêu chí khác nhau: **Shared Library** chỉ nên giữ mã hạ tầng cross-cutting đã ổn định (JWT, CORS, base response); **Shared Kernel** chỉ dành cho phần Domain Model mà nhiều Bounded Context cố ý đồng sở hữu và cùng quản trị — loại này càng ít càng tốt.

Đối chiếu nguyên tắc này với `kbm-shared` hiện tại cho thấy những phần cần tách:

Shared library cần tách

Với 23 tệp trong `kbm-shared`, KBM đang tạo ra sự ràng buộc (coupling) không cần thiết: mã nguồn đặc thù miền nghiệp vụ (domain-specific code) như `CompareUtil` nằm chung với các nghiệp vụ cắt ngang (cross-cutting concern) như JWT hay `GlobalExceptionHandler`. Theo nguyên lý tách biệt thiết kế trong [2a, Ch.2; 2b, Ch.4], chỉ các phần hạ tầng thực sự ổn định và dùng chung mới nên nằm trong thư viện nội bộ, còn hợp đồng API và định nghĩa sự kiện (event schema) cần trao đổi qua đặc tả độc lập ngôn ngữ (OpenAPI, JSON Schema — Chương 3, Chương 5) chứ không đóng gói chung vào một shared library dùng chéo giữa mọi dịch vụ. **Lộ trình chuyển đổi**:

1. Tách domain code về service riêng.
2. Chỉ giữ security và base response trong thư viện dùng chung.
3. Đưa domain enum (`TypeQuestion`, `AnswerStatus`) về context sở hữu.

Tuy nhiên, cần tránh những lầm tưởng kinh điển sau:

Sai lầm thường gặp

1. **Phân rã theo schema database** — Thiết kế service dựa trên các bảng CSDL thay vì nghiệp vụ. Hậu quả: tạo ra các CRUD API mỏng. *Phòng tránh*: Bắt đầu từ domain process (Event Storming §2.3), không từ schema.
2. **Chia quá nhỏ (nano-services)** — Tách mỗi entity thành 1 service riêng (UserService, RoleService, TokenService) khi chúng luôn thay đổi cùng nhau. Hậu quả: mỗi yêu cầu (request) cần gọi chéo 3–4 dịch vụ, dẫn đến độ trễ (latency) tăng cao và quá trình gỡ lỗi (debugging) trở nên phức tạp. *Phòng tránh*: tách theo Bounded Context, không theo entity. Nếu hai entity luôn thay đổi cùng nhau, chúng thuộc cùng service.
3. **Chia sẻ mã nguồn vì tiện, không vì thiết kế** — Đưa toàn bộ mã nguồn chung vào thư viện dùng chung mà không phân biệt cross-cutting concern và domain logic. Hậu quả: mỗi thay đổi trong thư viện dùng chung sẽ ảnh hưởng đến tất cả các dịch vụ, tạo ra sự phụ thuộc ngầm khi triển khai (deployment coupling). *Phòng tránh*: mỗi file trong thư viện dùng chung phải có phạm vi sử dụng và người chịu trách nhiệm rõ ràng, và các service dùng nó phải nâng phiên bản độc lập được; domain logic thì trả về context sở hữu (§2.5).

2.6. Dark Energy & Dark Matter — 10 lực chi phối việc phân tách service

Phần nâng cao (có thể lướt qua)

Người đọc mới có thể đọc lướt mục này ở lần đầu. Cốt lõi cần nhớ là cặp "lực đẩy / lực hút"; chi tiết 10 lực phát huy tác dụng khi thực sự phải cân nhắc tách hoặc gộp một service cụ thể.

Richardson trong phiên bản thứ hai (2025) giới thiệu một framework phân tích trade-off cho việc service decomposition, mượn ẩn dụ từ vật lý thiên văn:

- **Dark Energy — lực đẩy (repulsive)**: kéo các thành phần (component) ra xa nhau → nên phân tách thành dịch vụ riêng biệt.
- **Dark Matter — lực hút (attractive)**: kéo các thành phần lại gần nhau → nên giữ cùng service.

Bảng 2.7 liệt kê mười lực cụ thể và cách mỗi lực thể hiện trong KBM.

Bảng 2.7: *Mười lực Dark Energy / Dark Matter*

Lực	Loại	Mô tả	Ví dụ KBM
Simple interactions	Matter (hút)	Các thành phần tương tác phức tạp → giữ cùng service	`ContestQuestion` ↔ `Contest` luôn query chung → hợp lý ở cùng `kbm-core`
Efficient interactions	Matter (hút)	Yêu cầu độ trễ (latency) thấp → việc gọi hàm nội bộ (in-process) luôn nhanh hơn gọi qua mạng	Submission → Score cập nhật cần nhanh → hiện cùng service
Prefer ACID	Matter (hút)	Cần ACID transaction → cùng DB dễ hơn saga	Create contest + add questions = 1 transaction → cùng service
Minimize runtime coupling	Matter (hút)	Giảm thiểu sự phụ thuộc trong thời gian chạy (runtime dependency) dẫn đến giảm số lượng lệnh gọi qua mạng	Nếu tách phần tính điểm khỏi Core, mỗi lần nộp bài phải chờ thêm một lời gọi mạng tới service đó
Minimize design-time coupling	Matter (hút)	Giảm phụ thuộc lúc thiết kế (API/ schema chung): nếu hai bên tách ra mà vẫn phải sửa cùng nhau, coupling chỉ đổi hình thức	`kbm-shared` buộc các service Java rebuild đồng loạt — một dạng design-time coupling cần thu hẹp (§2.5)
Simple components	Energy (đẩy)	Mỗi service nhỏ, dễ hiểu	Mỗi service một codebase riêng, chỉ chứa code của context mình — dễ nắm hơn một khối gộp tất cả
Team autonomy	Energy (đẩy)	Các nhóm triển khai (deploy) độc lập, không cần phối hợp chéo (coordination)	Identity & Access và SQL Practice có thể triển khai độc lập; khi có nhiều nhóm, mỗi nhóm không phải chờ nhau
Fast deployment pipeline	Energy (đẩy)	Dịch vụ nhỏ gọn giúp quá trình biên dịch (build) và kiểm thử (test) diễn ra nhanh hơn	Build một service nhỏ nhanh hơn nhiều so với build cả khối
Support multiple tech stacks	Energy (đẩy)	Các dịch vụ khác nhau có thể sử dụng nền tảng công nghệ (tech stack) khác nhau	DevOps Practice viết bằng Go, phần còn lại của LMS bằng Java (§2.4)
Segregate by characteristics	Energy (đẩy)	Tách các phần có đặc tính vận hành khác biệt: nhu cầu mở rộng, tính sẵn sàng, yêu cầu bảo mật, dữ liệu sở hữu riêng…	Các dịch vụ Judge cần thêm tài nguyên khi có kỳ thi và cần sandbox bảo mật riêng — khác hẳn Identity & Access

Cách sử dụng: Khi phân vân "có nên tách component X thành service riêng?", liệt kê các lực tác động. Nếu Dark Energy (đẩy) chiếm ưu thế → tách. Nếu Dark Matter (hút) mạnh hơn → giữ chung. Đây không phải công thức chính xác mà là **framework tư duy** giúp cấu trúc cuộc thảo luận.

Dark Energy/Matter cho quyết định tách SQL Practice & Academic Management

KBM hiện có SQL Practice (`kbm-core`) và Academic Management (`kbm-academic`) dùng chung cơ sở dữ liệu. Phân tích:

- **Dark Matter (giữ chung)**: cơ sở dữ liệu dùng chung giúp đảm bảo ACID dễ dàng, bảng `assignment_questions` tham chiếu tới `questions.id` (tương tác đơn giản), nhóm phát triển rất nhỏ (1–2 lập trình viên chính)
- **Dark Energy (tách)**: domain logic khác biệt (bài tập vs kỳ thi), data ownership rõ ràng, tiềm năng triển khai độc lập

Kết luận: Dark Matter hiện mạnh hơn (do team nhỏ, shared DB). Trạng thái cân bằng sẽ dịch chuyển khi các tín hiệu quan sát được xuất hiện: nhóm phát triển tách thành nhiều team có ownership riêng, các thay đổi lockstep bắt đầu làm nghẽn pipeline, hoặc nhu cầu scale của hai domain khác hẳn nhau → lúc đó việc phân tách cơ sở dữ liệu và dịch vụ sẽ trở nên hợp lý hơn.

Đây chính là bài học thực tế của nguyên tắc không phân tách quá sớm (*premature decomposition* [4a, Ch.3; 4b, Ch.2]) khi các lực hút Dark Matter vẫn chiếm ưu thế (Richardson [2b, Ch.7]).

2.7. Assemblage Process — quy trình thiết kế service architecture

Phần nâng cao (có thể lướt qua)

Mục này tổng hợp các kỹ thuật phía trên thành một quy trình thiết kế hoàn chỉnh. Người đọc mới nên nắm chắc Bounded Context và Context Map trước; có thể quay lại quy trình tổng hợp này khi bắt tay thiết kế một hệ thống thật.

Richardson trong [2b, Ch.20] trình bày **Assemblage Process** — quy trình step-by-step tổng hợp các kỹ thuật đã học (Bounded Context, Context Map, Dark Energy/Matter) thành chuỗi bước đi từ yêu cầu nghiệp vụ đến kiến trúc microservices cụ thể:

1. **Khám phá System Operations** — Liệt kê các thao tác chính mà hệ thống phải hỗ trợ (ví dụ: `submitAnswer()`, `createContest()`, `gradeSubmission()`).
2. **Xác định Subdomains** — Phân vùng *không gian bài toán* thành các subdomain từ nghiệp vụ và system operations. Với KBM, các subdomain này về sau được ánh xạ thành bảy Bounded Context core (Identity & Access, SQL Practice, Network Practice, Code Practice, Academic Management, Examination & Proctoring, DevOps Practice) cùng các supporting context (Communication, Platform) trong không gian giải pháp (§2.4).
3. **Thiết kế Services & Collaborations** — Gán mỗi operation vào subdomain sở hữu logic của nó, nhóm các subdomain thành service bằng cách cân 10 lực Dark Energy/Dark Matter, rồi chọn mô hình giao tiếp giữa các service: đồng bộ REST/gRPC (Ch.4) hay bất đồng bộ Kafka (Ch.5).
4. **Đánh giá kiến trúc** — Đối chiếu kết quả với các quality attribute scenario (Ch.1): kiến trúc có đạt deployability, testability, các yêu cầu scale đã đặt ra không?
5. **Tái cấu trúc** — Quy trình không chạy một lần. Khi kiến thức domain sâu hơn hoặc phép đánh giá phát hiện lệch, ranh giới service được điều chỉnh và vòng lặp tiếp tục.

Assemblage process biến quá trình thiết kế kiến trúc — vốn mang tính nghệ thuật và chủ quan — thành chuỗi các quyết định kỹ thuật có thể giải thích được. Thay vì nói "tách Judge Service vì có vẻ hợp lý", chúng ta có thể lập luận: *"Judge Service được tách vì các lực đẩy Dark Energy (Segregate by characteristics — cần cô lập sandbox và mở rộng độc lập khi thi, cùng Simple components và Fast deployment pipeline) vượt trội so với lực hút Dark Matter (Efficient interactions) trong ngữ cảnh này."*

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Bản đồ Bounded Context (Context Map)**: context-map.html

2.8. Tổng kết

Quy trình phân tách hệ thống trải dài từ cấp vĩ mô (Conway's Law) đến vi mô (Entity, Aggregate, Repository).

Conway's Law nhắc nhở rằng kiến trúc phần mềm phản ánh cấu trúc tổ chức và chịu ràng buộc từ cấu trúc đó. Inverse Conway Maneuver cho phép chúng ta chủ động thiết kế team để đạt kiến trúc mong muốn.

Chúng ta có hai phương pháp bổ trợ để xác định ranh giới service. **DDD** cung cấp ngôn ngữ và heuristics — Bounded Context, Context Map, Event Storming — phù hợp khi team hiểu domain sâu. **Erl's Service-Oriented Analysis** cung cấp quy trình step-by-step — từ automation candidates đến service layers — phù hợp khi team cần một framework có cấu trúc. Cả hai đều dẫn đến ranh giới tương đồng (như minh họa với KBM qua năm nhóm dịch vụ tiêu biểu ở bước tổng hợp của Erl), nhưng xuất phát từ góc nhìn khác nhau. Dark Energy/Dark Matter forces làm tường minh các yếu tố ảnh hưởng đến quyết định tách/gộp, và Assemblage Process (§2.7) tổng hợp tất cả thành quy trình thiết kế có hệ thống.

Phân tích KBM cho thấy 7 Bounded Context core rõ ràng (Identity & Access, SQL Practice, Network Practice, Code Practice, Academic Management, Examination & Proctoring, DevOps Practice) cùng 2 supporting context (Communication và Platform / Infrastructure), mỗi context tương ứng với một nhóm service hoặc binary (dù về mặt triển khai vật lý, phần điều phối thi của Examination & Proctoring hiện vẫn nằm trong `kbm-core` với mục tiêu tách thành `kbm-exam`). Thư viện dùng chung của KBM — dù tiện lợi cho team nhỏ — cần được quản lý có chủ đích, phân biệt rõ giữa cross-cutting concern của Shared Library (nên share) và domain logic (chỉ share qua Shared Kernel khi các context thực sự đồng sở hữu).

2.9. Đọc thêm

Sách tham khảo chính:

1. [1] Thomas Erl, *Service-Oriented Architecture: Analysis and Design for Services and Microservices*, 2nd Ed. — Service-Oriented Analysis, Service Layers (task/entity/utility), Service Inventory Blueprint
2. [6] Eric Evans, *Domain-Driven Design* — Ch.1–5: Knowledge Crunching, Ubiquitous Language, Building Blocks; Ch.14: Bounded Context, Context Map
3. [2a] Chris Richardson, *Microservices Patterns*, 1st Ed. — Ch.2: Decomposition Strategies
4. [2b] Chris Richardson, *Microservices Patterns*, 2nd Ed. — Ch.4: Loose Coupling, Dark Energy/Dark Matter Forces; Ch.7: Microservice Architecture; Ch.20: Assemblage Process
5. [4a] Sam Newman, *Building Microservices* — Ch.3: How to Model Services; Ch.10: Conway's Law
6. [4b] Sam Newman, *Monolith to Microservices* — Ch.2: Planning a Migration, Decomposition by Domain
7. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Ch.3: Service Boundaries, DDD/Bounded Context

Sách bổ trợ:

8. Vaughn Vernon, *Implementing Domain-Driven Design* (2013) — Ch.2: Strategic Design with Bounded Contexts
9. Matthew Skelton & Manuel Pais, *Team Topologies* (2019) — Four Team Types, Cognitive Load, Conway's Law
10. Alberto Brandolini, *Introducing EventStorming* (2013, Leanpub, cập nhật liên tục) — Domain discovery workshops

Nguồn trực tuyến:

- MacCormack, Rusnak & Baldwin, *"Exploring the Duality between Product and Organizational Architectures"* (2008) — Harvard Business School Working Paper

Bài tập: 2-0, 2-1, 2-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[1] Erl, T. (2016). *Service-Oriented Architecture: Analysis and Design for Services and Microservices* (2nd ed.). Prentice Hall.

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[2b] Richardson, C. (2025). *Microservices Patterns* (2nd ed.; bản MEAP, truy cập 07/2026). Manning Publications.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.

[6] Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.